PRT582

Software Unit Testing Report

Intelligent Timetable Scheduler

Student	Romik Gurung
Student ID	[INSERT STUDENT ID]
Unit	PRT582
AI tools used	Claude (main development assistant); ChatGPT (report consolidation)

Development approach: AI-assisted Test-Driven Development (AI-TDD)

Submission Completion Checklist

This report is grounded in the assignment brief, the supplied timetable scheduler source code and tests, the Iterations.docx screenshots, and the 40+ screenshots of the Claude conversation supplied for this report. The only remaining details that cannot be verified from the supplied material are the student ID and final GitHub repository URL.

- Insert student ID on the cover page.
- Create/push the final GitHub repository and insert its URL in Section 8.
- Make sure the repository contains the same final code and tests used for the 134-pass result.
- Run the final test and coverage commands immediately before submission.
- Submit the report together with source code and automated unit tests in the required ZIP.

1. Introduction

This project develops an Intelligent Timetable Scheduler for a university teaching week. The scheduler assigns teaching sessions to rooms and time slots while satisfying hard constraints such as room capacity, lecturer availability, no clashes and prerequisite ordering. It also uses soft preferences to improve timetable quality, such as preferred time windows and choosing the smallest suitable room.

The project was completed using an AI-assisted Test-Driven Development workflow. Claude was the main AI development assistant. The process did not begin perfectly: an initial version was generated too broadly in one step. After reviewing the assignment requirements, I recognised that this did not demonstrate the required iterative TDD process. I therefore restarted the work as seven smaller RED-GREEN iterations, running the tests at each stage and keeping screenshots of the results. This report documents that iterative rebuild honestly rather than presenting the first prototype as if it had been developed test-first.

The final implementation uses depth-first backtracking. A simpler greedy first-fit scheduler was deliberately tested first and appeared correct because its current test suite was green. A later regression test exposed a case where the greedy algorithm made an early legal choice that blocked a valid timetable. The algorithm was then replaced with backtracking so it could undo decisions and try alternatives.

The final project contains 134 automated pytest tests. In the supplied final version, all 134 tests pass and the timetable package reaches 100% statement coverage.

1.1 Assessment Evidence Overview

The report is organised to match the five assessment areas shown in the marking rubric. Each area is supported by concrete evidence from the requirements, automated tests, Claude development transcript, defect analysis, final execution results and submission artefacts.

Rubric area	Where addressed	Evidence in this report
Requirements Analysis & Test Design	Sections 2-3	Functional/non-functional requirements, assumptions, constraints, 10+ expected behaviours, boundaries, invalid inputs, and justified test groups.
AI-Assisted Development Process	Section 4	Seven RED-GREEN iterations, actual interaction summaries, AI response summaries, accept/modify/reject decisions, reasons, and executed-test evidence.
Critical Evaluation & Improvement of AI Output	Section 5	Real defects found after initially green states, before/after fixes, regression tests, and evaluation of limitations in AI-generated tests.
Software Quality & Automated Testing	Sections 5.5-6	Modular design, deterministic backtracking, error handling, 134 passing tests, regression coverage, and 100% statement coverage.
Report, Evidence & Professional Presentation	Sections 6-8 + Appendix	Captioned iteration evidence, final test/coverage evidence, repository/submission checklist, and a traceable evidence map.

2. Requirements Analysis

2.1 Functional Requirements

- Create and validate teaching time slots for Monday to Friday within supported teaching hours.
- Represent rooms with a unique identifier and positive seating capacity.
- Represent lecturers with identifiers, names and optional availability windows.
- Represent teaching sessions with unit code, lecturer, enrolment, duration, student group and optional preferred time windows.
- Assign every session to exactly one legal room and time slot when a feasible timetable exists.
- Ensure room capacity is at least the session enrolment.
- Ensure session duration matches the selected time-slot duration.
- Ensure the lecturer is available for the complete selected slot.
- Prevent lecturer clashes, room clashes and student-group clashes.
- Enforce prerequisite ordering so prerequisite units occur before dependent units.
- Prefer requested time windows where possible without breaking hard constraints.
- Prefer the smallest suitable room to reduce waste of large rooms.
- Raise a clear error when no feasible timetable exists or the search budget is exceeded.
- Produce a readable timetable and diagnostic explanation for candidate placements.

2.2 Non-Functional Requirements

- Correctness: every returned timetable must satisfy all hard constraints.
- Maintainability: domain models, constraint functions and the search algorithm are separated into modules.
- Testability: important scheduling rules can be tested independently.
- Determinism: identical valid input should produce the same timetable on repeated runs.
- Usability: invalid inputs and infeasible schedules should raise meaningful errors.
- Performance protection: a maximum search-step budget prevents unbounded backtracking.

- Portability: the application runs in Python and uses pytest/pytest-cov for development testing.

2.3 Assumptions

- Teaching occurs Monday to Friday.
- Time is represented in whole hours.
- The supported teaching day is 08:00 to 20:00.
- A lecturer with no stated availability is treated as unrestricted.
- A session must exactly fit one generated time slot.
- Back-to-back classes are allowed because touching time slots do not overlap.
- Prerequisite ordering is based on position in the teaching week.
- Soft preferences may be ignored when necessary to satisfy hard constraints.

2.4 Constraints

- Room capacity must be greater than or equal to enrolment.
- Lecturers can only teach within their availability and cannot teach overlapping sessions.
- A room cannot host overlapping sessions.
- A student group cannot attend overlapping sessions.
- Prerequisite units must begin strictly before the dependent unit.
- Circular and self-referential prerequisite definitions are invalid.
- Preferred times and room efficiency are soft constraints only.
- The scheduler stops if its configured search-step budget is exhausted.

2.5 Expected System Behaviours

1. A lower-case day such as "mon" is normalised to "MON".
2. Adjacent slots such as 09:00-11:00 and 11:00-13:00 do not overlap.
3. A room with exactly the required number of seats is accepted.
4. A room one seat too small is rejected.
5. A lecturer may teach back-to-back sessions but not overlapping sessions.
6. A room may be reused in a later non-overlapping slot.
7. A student group cannot be scheduled into two simultaneous sessions.
8. A lecturer with no availability restrictions can use any valid supplied slot.
9. A preferred slot is chosen when it is legal and available.
10. A preference is ignored if using it would break a hard constraint.
11. A large class is assigned only to a room with enough capacity.
12. A small class should use a smaller suitable room when possible.
13. A dependent unit is scheduled after its prerequisite.
14. A circular prerequisite relationship is rejected.
15. The scheduler backtracks when an early legal choice blocks a later session.
16. Repeated solve() calls reset internal search state and remain deterministic.

2.6 Boundary Conditions

- Earliest and latest legal teaching hours (08:00 and 20:00 boundaries).
- Room capacity exactly equal to enrolment and one seat below enrolment.
- Back-to-back slots that touch at the boundary but do not overlap.
- Prerequisite sessions that start at the same time, later on the same day, or on a later day.

- Empty availability meaning unrestricted availability.
- Repeated calls to solve() on the same Scheduler object.
- Search step count exactly at or beyond the configured maximum.

2.7 Invalid Input and Exceptional Scenarios

- Unknown day names, non-integer hours, zero-length slots and reversed start/end times.
- Empty or non-string identifiers for rooms, lecturers or sessions.
- Zero, negative or non-integer capacity, enrolment or duration.
- No sessions, no rooms or no time slots supplied.
- Duplicate session IDs or room IDs.
- A session referencing an unknown lecturer.
- Circular or self-referential prerequisites.
- A session too large for every available room.
- A lecturer whose availability cannot fit the session duration.
- More conflicting sessions than the available resources can support.
- A search problem exceeding the maximum step budget.

3. Test Design

The test suite was designed in groups so that low-level behaviours were checked before the full scheduling algorithm. Model tests cover validation and time behaviour, constraint tests cover individual scheduling rules, and scheduler tests cover complete end-to-end behaviour, regression cases and exceptional situations. This separation made it easier to identify whether a failure came from input validation, a rule, or the search algorithm.

3.1 Major Test Groups and Purpose

Test group	Behaviour tested	Why necessary	Defect prevented
TimeSlot validation	Day/hour validation, containment and overlap boundaries	All scheduling rules depend on correct time behaviour	Invalid hours, unknown days, and treating touching slots as clashes
Room/Lecturer/Session models	Identifiers, capacities, enrolment, duration, availability and preferences	Reject malformed domain data before search	Invalid IDs, bool accepted as int, invalid capacities/durations
Hard constraints	Capacity, duration, availability, lecturer/room/group clashes	Each rule can be proven independently	A single combined scheduler hiding which rule is wrong
Scheduler integration	Every session is placed legally or a clear error is raised	Checks that rules still work when combined	Integration defects that isolated unit tests miss
Backtracking regression	Bad early room/time choice must be recoverable	A legal local choice may be globally wrong	Greedy first-fit falsely reporting no timetable
Prerequisites	Strict before-ordering, both directions, cycle detection	Search order is not chronological	Dependent unit placed before/same time as prerequisite
Soft constraints	Preferred slots and smallest-room-first ordering	Improve timetable quality without rejecting feasible solutions	Preferences accidentally behaving as hard constraints
Robustness/NFR tests	Determinism, max steps, repeated solve, explain()	Protect state and performance behaviour	Stale step counter, nondeterministic output, endless search

3.2 Final Test Suite Summary

The final suite contains 134 pytest tests across tests/test_models.py, tests/test_constraints.py and tests/test_scheduler.py. The suite includes normal behaviour, boundary conditions, invalid input, exceptions, characterisation tests and regression tests. Coverage of the final timetable package is 100% statement coverage.

4. AI-Assisted Development Process

Claude was the main AI development assistant. I first asked for help understanding and completing the assignment in simple language. After selecting the timetable scheduler, Claude generated an initial complete solution too quickly. I then noticed that the brief required an iterative development process and told Claude: "Just like it is mentioned in the document lets do everything step by step, as it says don't finish everything in one go." Claude agreed that the earlier approach was too broad and restarted the work as seven RED-GREEN iterations.

The transcript shows a useful change in how I used AI. Instead of asking for the whole application, I ran each provided RED state, sent the result back, reviewed the explanation, then moved to the GREEN implementation. In later iterations I also used the tests to challenge AI-generated logic rather than assuming a green suite meant the design was correct.

Important integrity note: the Claude conversation also discussed Git commits. Commands for per-iteration commits were suggested, but the supplied ZIP does not prove those commits were actually made. Therefore this report does not claim a verified seven-commit Git history. The RED/GREEN screenshots are used as the main development-process evidence, and the final GitHub URL must be added before submission.

4.1 AI Interaction Evidence Matrix

The table below makes the AI-TDD evidence explicit. It uses only interactions visible in the supplied Claude screenshots. Where the user response was a screenshot/result rather than a written prompt, it is described as a continuation interaction rather than inventing wording that was not shown.

Iteration	User interaction / prompt context	AI response summary	Decision	Reason	Executed-test evidence
1 - TimeSlot	Asked for step-by-step work after choosing the timetable scheduler; Claude then introduced Iteration 1 RED tests.	Generated TimeSlot tests and a deliberately incomplete skeleton.	Accepted with review	The tests covered validation, boundaries and overlap behaviour; implementation was added only after RED was observed.	16 failed / 5 passed -> 21 passed
2 - Room/Lecturer/Session	Continued after reporting Iteration 1 GREEN; Claude moved to Room, Lecturer and Session RED tests.	Added model tests/skeletons and explained availability/preference design decisions.	Accepted + refactored	Shared identifier validation was refactored to avoid duplication; regression tests from Iteration 1 remained green.	21 failed / 28 passed -> 49 passed
3 - Constraint rules	Continued after 49 passed; Claude introduced independent hard-constraint tests.	Generated constraint tests, a constraint module skeleton and Assignment model.	Accepted	Independent rule functions made failures easier to isolate before the	14 failed / 59 passed -> 73 passed

Iteration	User interaction / prompt context	AI response summary	Decision	Reason	Executed-test evidence
				scheduler existed.	
4 - Scheduler	After 73 passed, Claude introduced scheduler tests, then a greedy first-fit implementation and challenged it with a counterexample.	Produced a working greedy scheduler first, then a regression case and backtracking rewrite.	Initially accepted, then rejected/modified	The greedy version passed 100 tests but failed a valid timetable case; backtracking was required for correctness.	22 failed / 78 passed -> 100 passed -> 1 failed / 102 passed -> 103 passed
5 - Prerequisites	Continued to prerequisite ordering and cycle detection.	Generated direct prerequisite tests and an initial one-direction implementation.	Modified	All direct rule tests passed, but end-to-end scheduler tests exposed a search-order integration defect.	7 failed / 109 passed -> 2 failed / 114 passed -> 116 passed
6 - Soft constraints	Continued to preferred slots and room-efficiency tests.	Implemented preferences as candidate ordering rather than legality filters.	Accepted	This preserved feasibility because soft constraints could never override hard constraints.	4 failed / 121 passed -> 125 passed
7 - Budget/determinism/coverage	Continued to deferred search-budget, determinism, explain() and coverage work; user selected deletion of unused Assignment.__str__.	Added budget/diagnostics; coverage exposed one unused statement; Claude offered delete-or-test options.	Modified	Dead code was removed instead of adding an artificial test only to increase the percentage.	6 failed / 128 passed -> 134 passed -> 99% -> 100% coverage

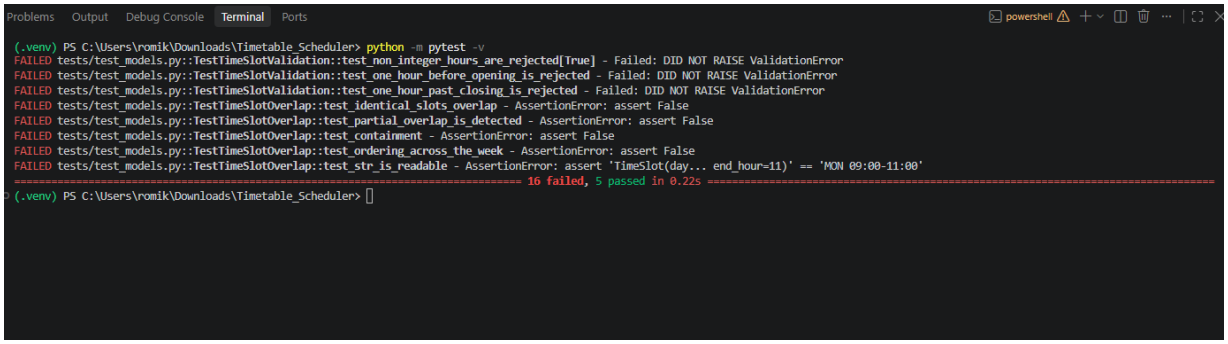
4.2 Iteration 1 - TimeSlot validation and overlap detection

Prompt / interaction: Initial direction: “Help me with this, (simple language)” followed by selecting “Timetable scheduler” and asking for all required parts. After restarting iteratively, Claude prepared TimeSlot tests first and asked me to run them RED before receiving the implementation.

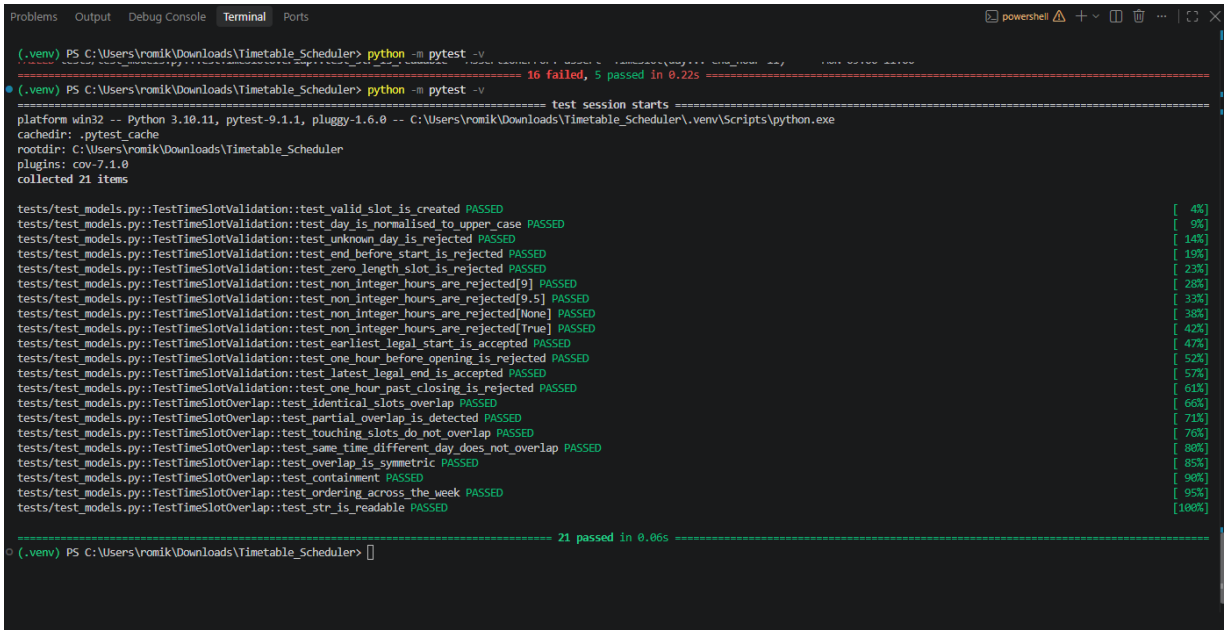
AI response summary: Claude created tests for valid/invalid days and hours, normalisation, containment and overlap behaviour, with a skeleton TimeSlot implementation that intentionally failed the new behaviours.

Decision and justification: Accepted with review. I ran the RED suite (16 failed, 5 passed), then replaced the skeleton with the GREEN implementation. The final result for this iteration was 21 passed.

What I learned / reviewed: A key edge case was that Python bool is a subclass of int. Numeric validation therefore had to reject bool explicitly. Another important boundary was using < rather than <= for overlap so 09:00-11:00 and 11:00-13:00 remain legal back-to-back slots.



Iteration 1 RED: 16 failed, 5 passed.



Iteration 1 GREEN: 21 passed.

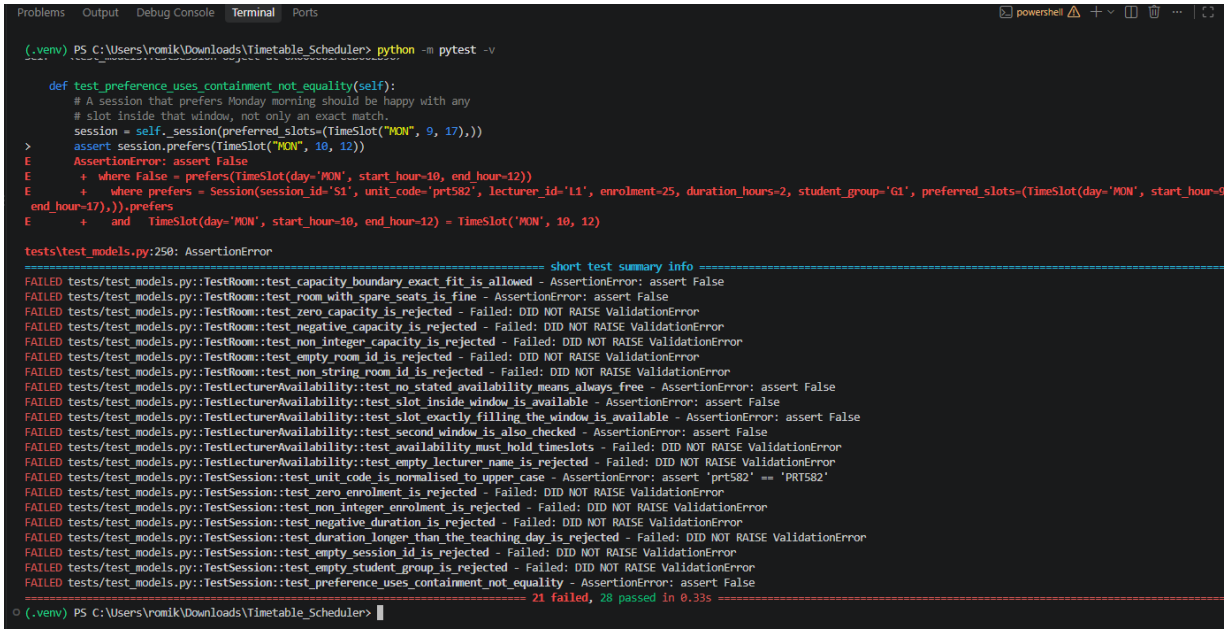
4.3 Iteration 2 - Room, Lecturer and Session

Prompt / interaction: After I sent the successful Iteration 1 result, the conversation continued to “Iteration 2: Room, Lecturer and Session. Red phase first, same pattern.” I ran the supplied RED tests before asking for the GREEN implementation.

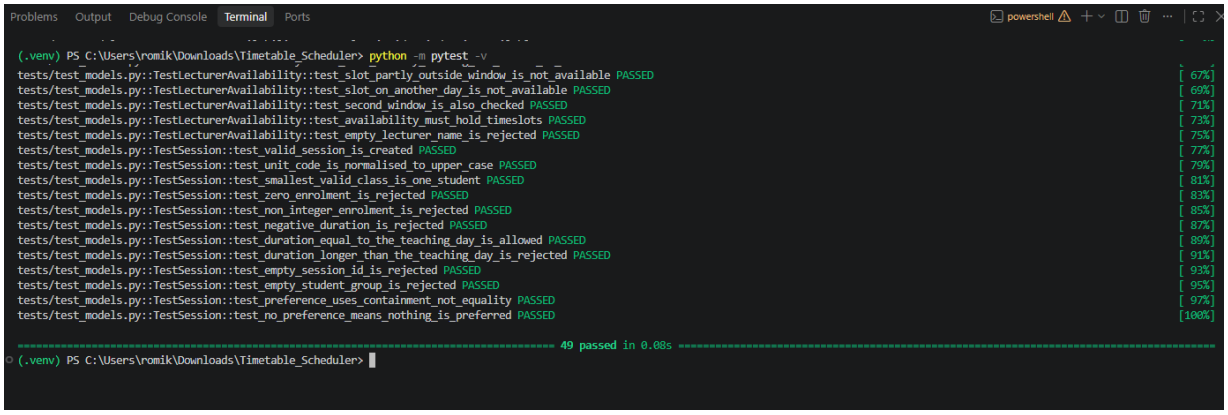
AI response summary: Claude added tests and skeleton classes for room capacity, lecturer availability, session validation and preference containment. It highlighted design choices such as treating empty lecturer availability as unrestricted.

Decision and justification: Accepted and refactored. RED produced 21 failed and 28 passed. GREEN produced 49 passed. Shared identifier validation was refactored into one helper rather than duplicated across the three classes.

What I learned / reviewed: The iteration demonstrated regression protection: all 21 TimeSlot tests from Iteration 1 continued to pass while new model tests were failing. It also clarified that a class must fit completely inside a lecturer availability window.



Iteration 2 RED: 21 failed, 28 passed.



Iteration 2 GREEN: 49 passed.

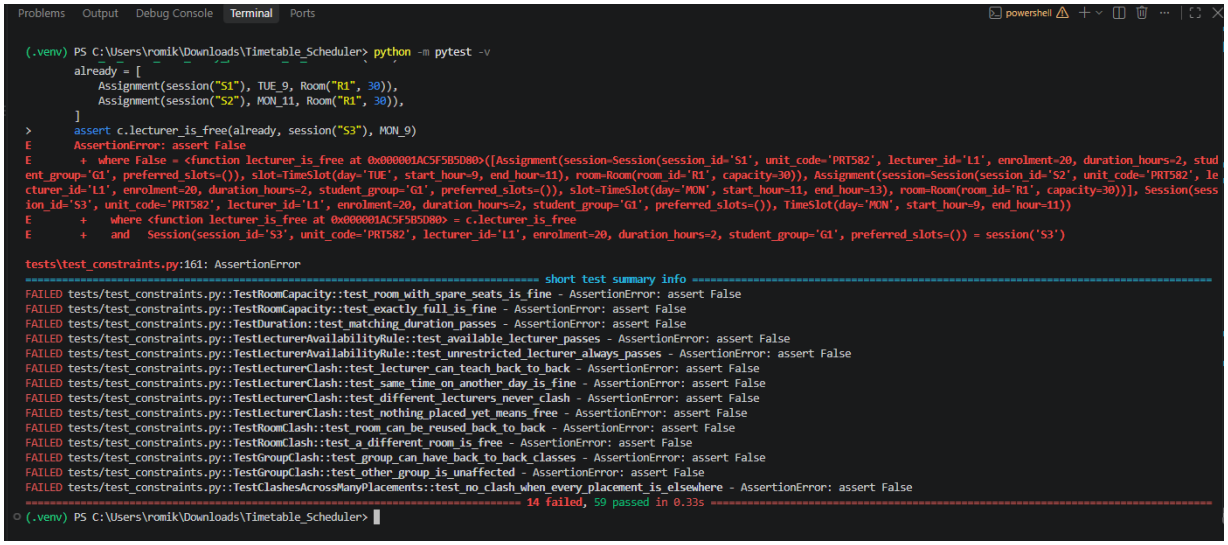
4.4 Iteration 3 - Independent constraint rules

Prompt / interaction: After the model layer was green, Claude moved to “Iteration 3: the constraint rules” and supplied tests for capacity, duration, lecturer availability and three clash rules before implementation.

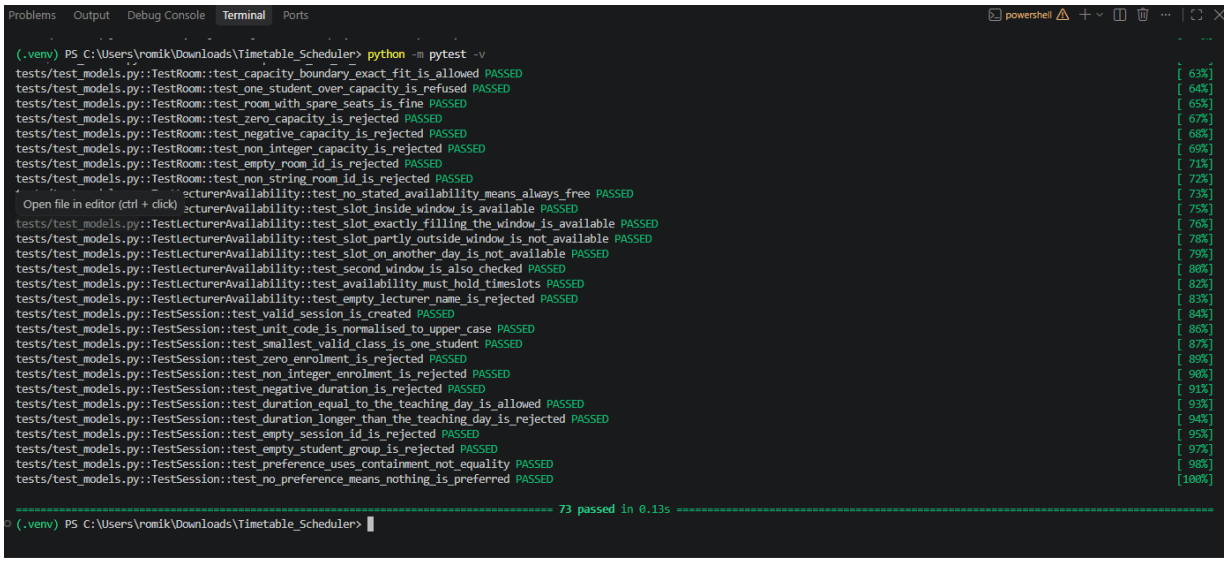
AI response summary: Claude created tests/test_constraints.py, a skeleton constraints.py, and the Assignment model. The rule functions were designed to accept existing assignments directly rather than depend on the Scheduler.

Decision and justification: Accepted. RED produced 14 failed and 59 passed; GREEN produced 73 passed. I kept the constraint rules separated because this made failures easier to locate and let the rules be tested before any scheduling algorithm existed.

What I learned / reviewed: Some negative-direction tests passed even against stub functions returning False. This was an important testing lesson: a passing test is not automatically strong evidence. Paired positive and negative tests were needed to make the rule behaviour meaningful.



Iteration 3 RED: 14 failed, 59 passed.



Iteration 3 GREEN: 73 passed.

4.5 Iteration 4 - Scheduler, greedy defect and backtracking fix

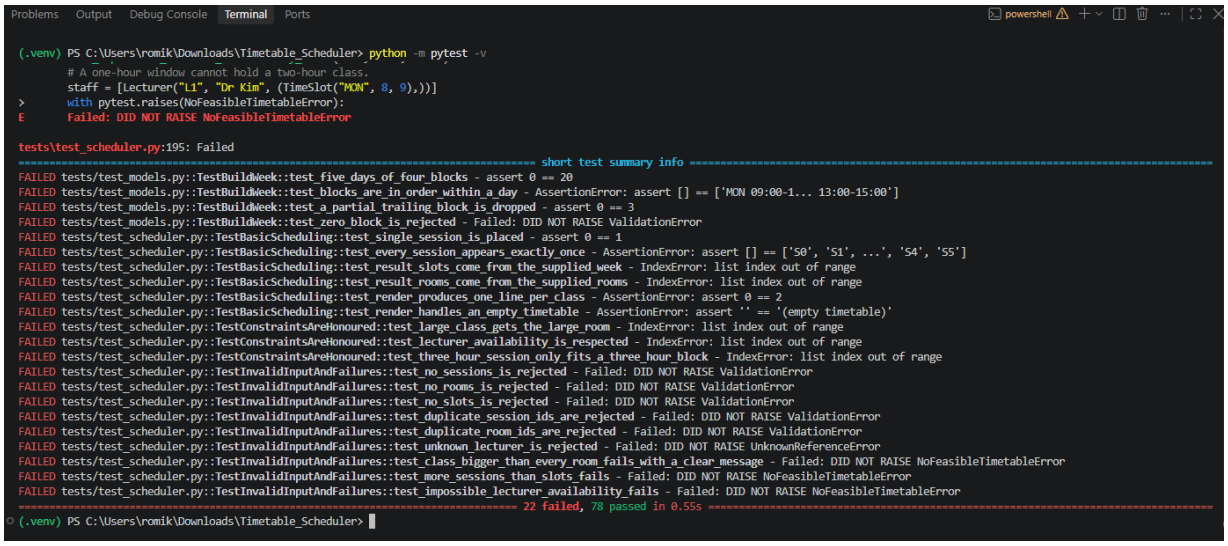
Prompt / interaction: Claude introduced the scheduler in three stages. First it supplied RED scheduler tests. After those failed, it supplied a simple greedy first-fit implementation. It then asked me to reason about whether an early legal room choice could make a later session impossible.

AI response summary: The initial scheduler skeleton produced 22 failed and 78 passed. The greedy implementation then produced 100 passed. Claude then described a two-room/two-class case in which SMALL_CLASS takes the only large room first and BIG_CLASS cannot be placed, even though a valid timetable exists.

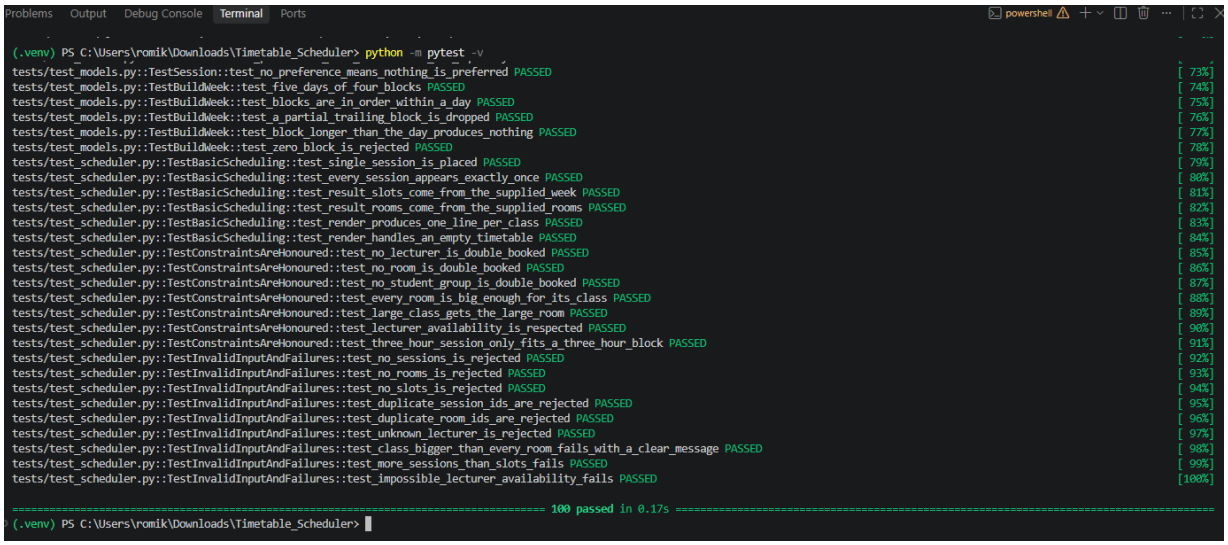
Decision and justification: The greedy version was rejected after a new regression test exposed the defect. The new RED state was 1 failed and 102 passed. Claude then replaced first-fit with depth-first backtracking, producing 103 passed.

What I learned / reviewed: This was the most important defect in the project. The green suite at 100 tests did not prove the algorithm was generally correct; it only proved the tested behaviours. A new test was needed to

expose the missing scenario. Backtracking fixed the problem by undoing a placement with placed.pop() and trying another candidate.

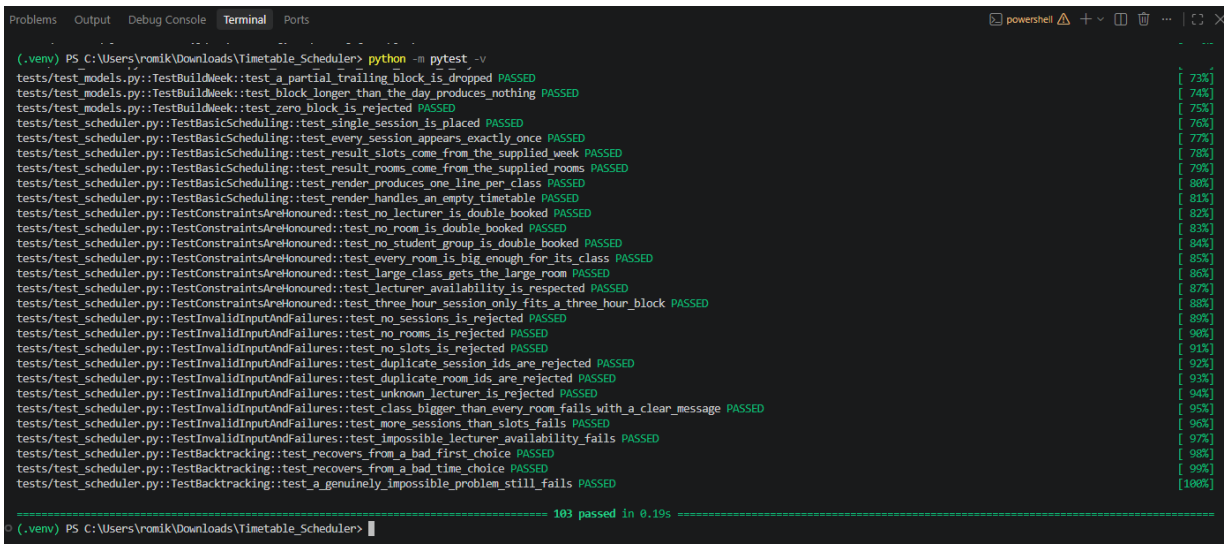


Iteration 4A RED: scheduler tests added - 22 failed, 78 passed.



Iteration 4B GREEN: greedy first-fit - 100 passed.

Iteration 4C RED again: regression test exposes greedy defect - 1 failed, 102 passed.



Iteration 4C GREEN: backtracking fix - 103 passed.

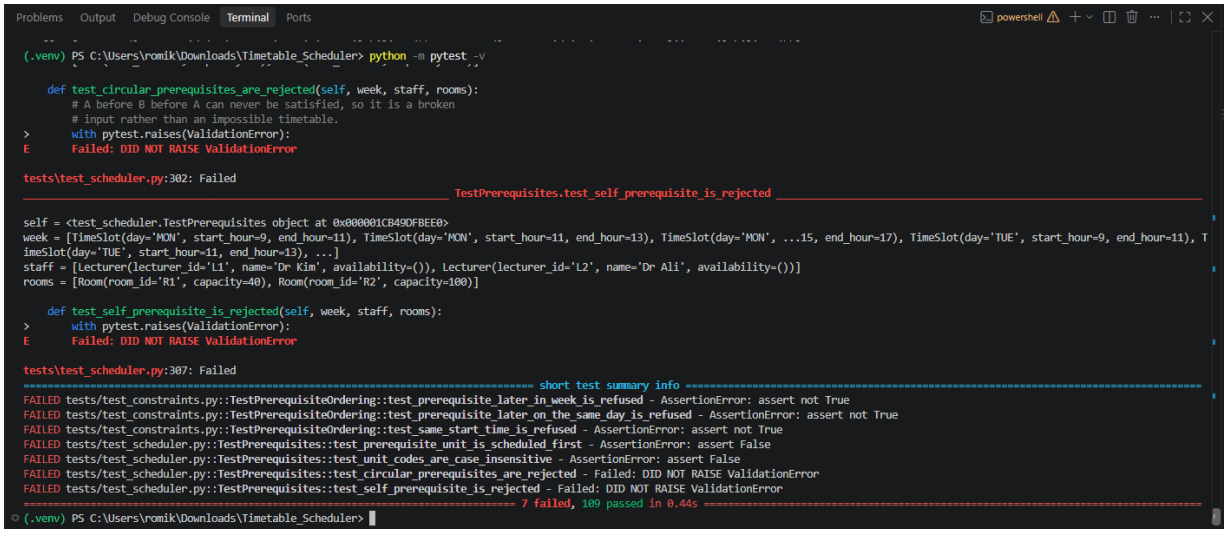
4.6 Iteration 5 - Prerequisite ordering

Prompt / interaction: Claude next added prerequisite-ordering and cycle-detection tests. The RED expectation was 7 failed and 109 passed. After the first implementation it specifically told me not to fix the remaining failures immediately, but to inspect why isolated constraint tests passed while scheduler tests failed.

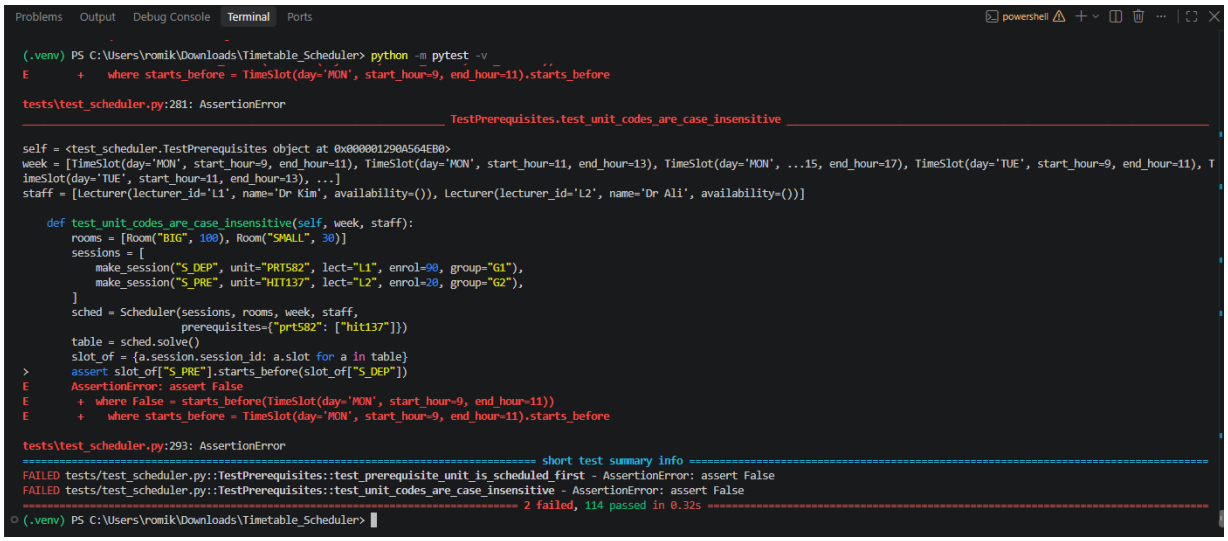
AI response summary: The first prerequisite implementation checked only whether an already-placed session was a prerequisite of the current one. All seven direct rule tests passed, but two end-to-end scheduler tests still failed because the search places sessions in “hardest first” order rather than chronological order.

Decision and justification: Modified. I accepted the basic rule but the one-direction assumption was incomplete. The fix checked both directions: whether an already-placed session is my prerequisite, and whether I am a prerequisite of an already-placed session. Final result: 116 passed.

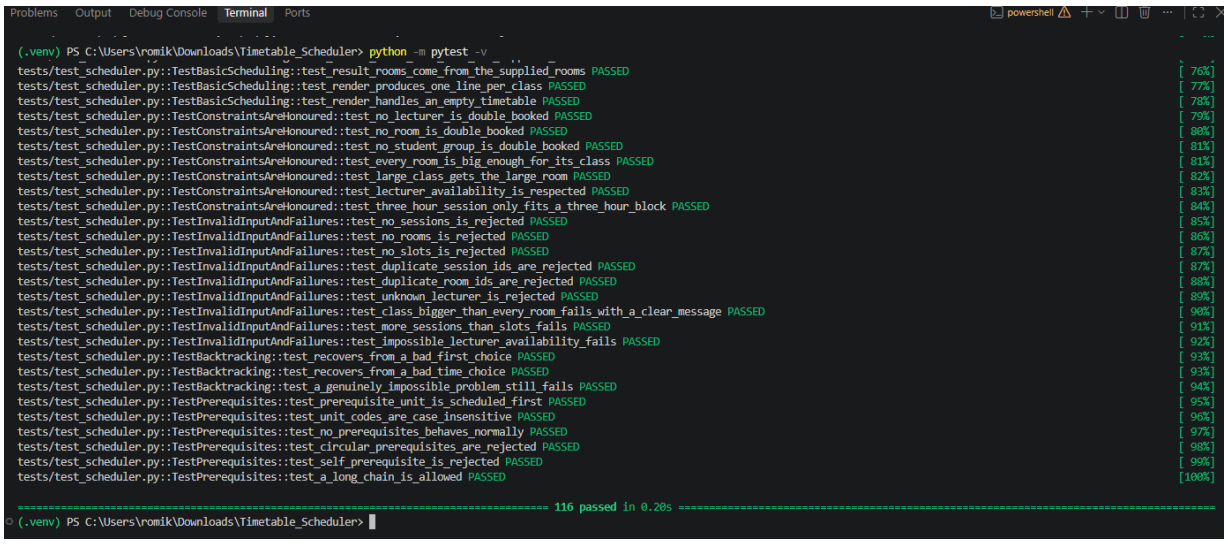
What I learned / reviewed: This defect showed why unit tests alone are not enough. A rule can pass in isolation and still fail when combined with a search algorithm that uses a different ordering. Integration-style scheduler tests caught the problem.



Iteration 5A RED: prerequisite tests added - 7 failed, 109 passed.



Iteration 5B partial GREEN: 2 failed, 114 passed; integration defect remains.



Iteration 5C GREEN: prerequisite check fixed in both directions - 116 passed.

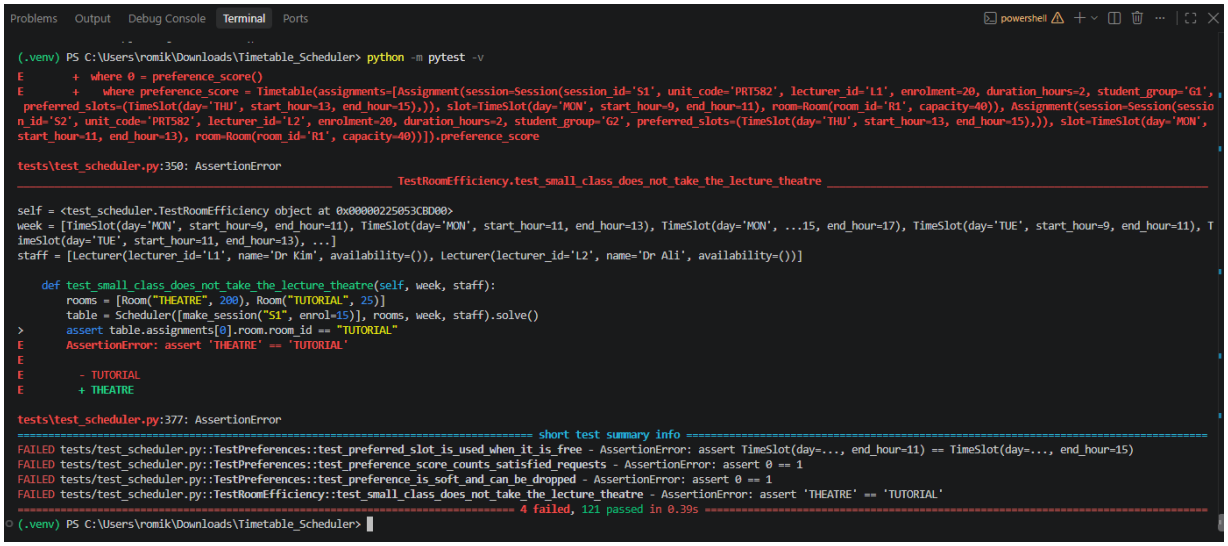
4.7 Iteration 6 - Preferred time slots and room efficiency

Prompt / interaction: Claude introduced two soft constraints: preferred time windows and avoiding waste of large rooms. The RED state expected 4 failed and 121 passed.

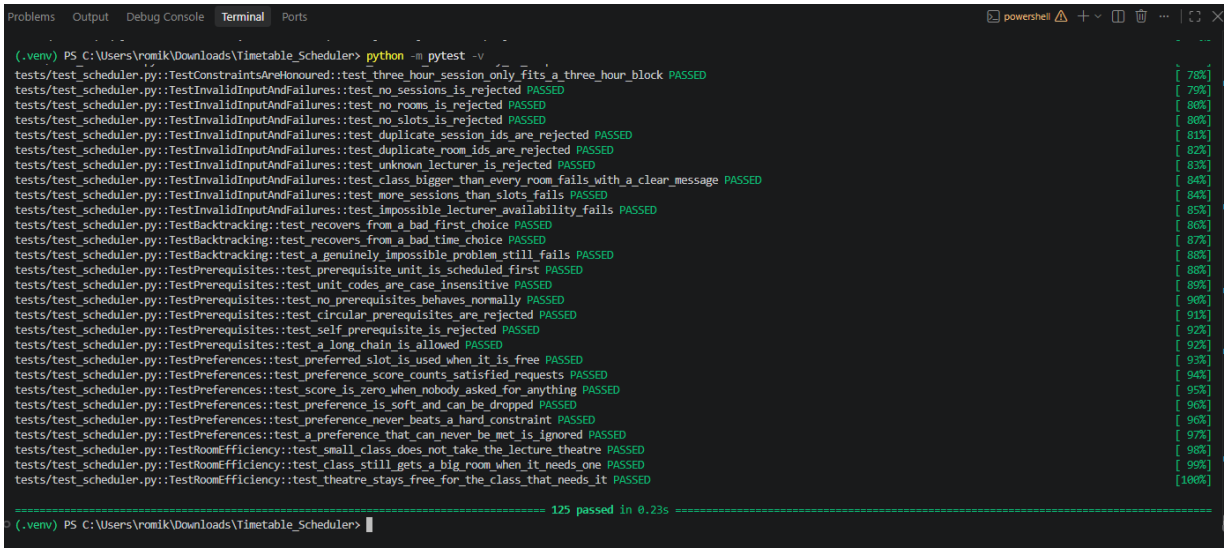
AI response summary: Claude explained that soft constraints should change the order in which legal candidates are tried, not remove candidates. The implementation sorted candidates so preferred slots and smaller suitable rooms are attempted first while all hard constraints remain decisive.

Decision and justification: Accepted. RED produced 4 failed and 121 passed. GREEN produced 125 passed. No new defect was discovered in this iteration, and I recorded that honestly instead of inventing one.

What I learned / reviewed: A preference must never make a feasible timetable impossible. This is why candidate sorting is safer than filtering. The tests also showed again that some new tests can pass before implementation because they characterise behaviour already guaranteed by hard constraints.



Iteration 6A RED: 4 failed, 121 passed.



Iteration 6B GREEN: 125 passed.

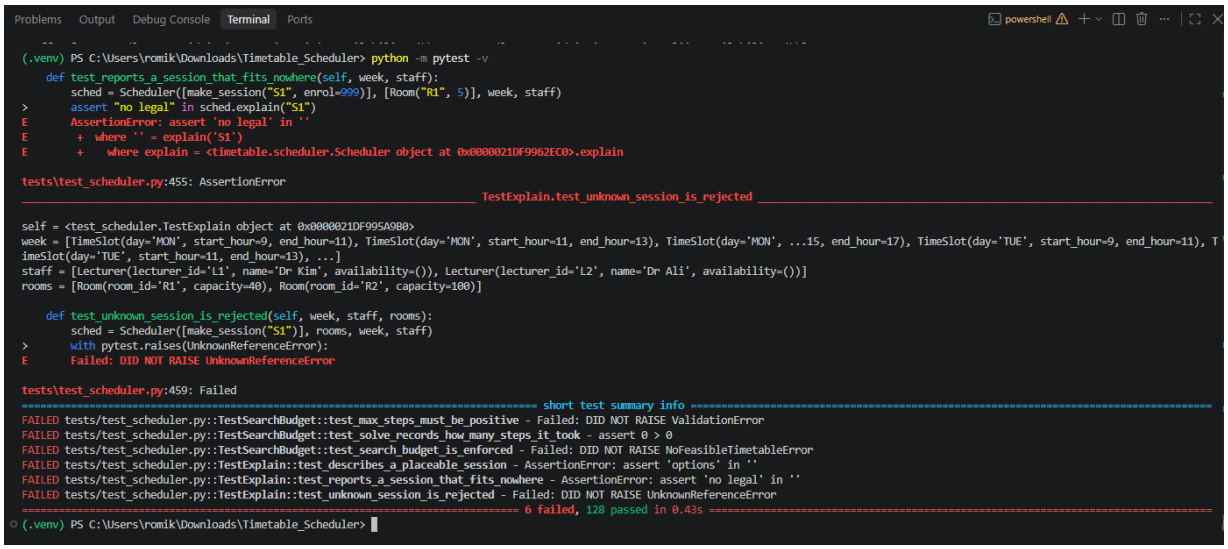
4.8 Iteration 7 - Search budget, determinism, diagnostics and coverage cleanup

Prompt / interaction: The final iteration added the deferred search-step budget, determinism tests and explain() diagnostics. Claude stated that three determinism tests already passed because previous tie-break ordering had already made the implementation deterministic.

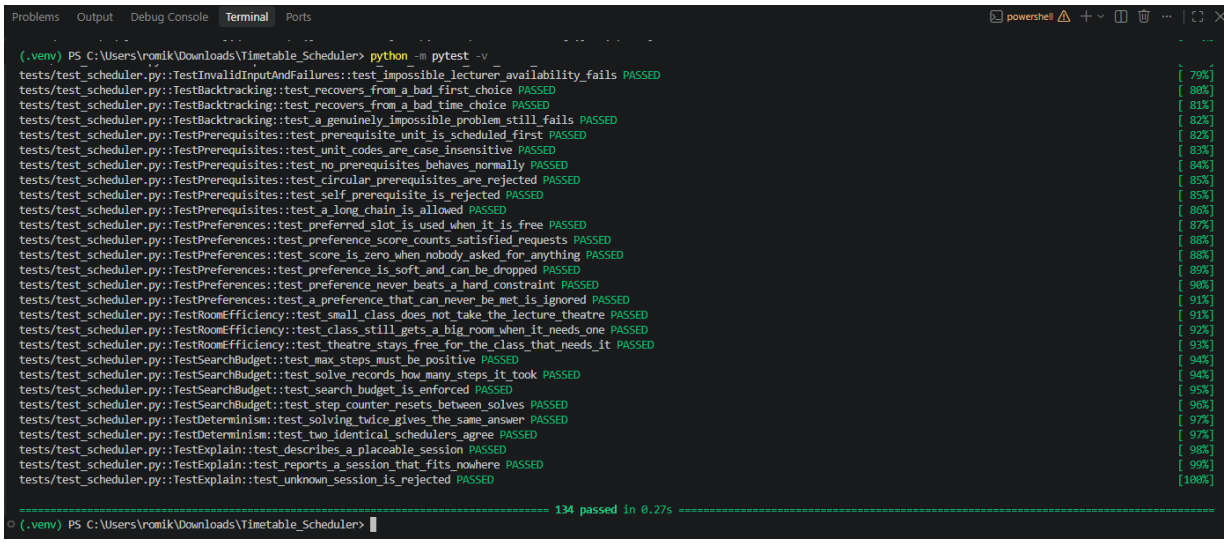
AI response summary: The RED state produced 6 failed and 128 passed. The GREEN implementation added the search budget and diagnostics, producing 134 passed. Running coverage then showed 99% with one unexecuted Assignment.__str__ line.

Decision and justification: Modified. When asked what to do with Assignment.__str__, I selected “Delete it as dead code.” Claude removed the unused method, added package exports, README and demo app. The final run produced 134 passed, 100% coverage, and the demo timetable in 11 search steps.

What I learned / reviewed: Coverage was used as a diagnostic, not just a target. The missing line was dead code, so removing it was cleaner than writing an artificial test only to make the percentage reach 100%.



Iteration 7A RED: 6 failed, 128 passed.



Iteration 7B GREEN: 134 passed before final coverage cleanup.

5. Evaluation and Improvement of AI Output

The assignment required critical evaluation of both AI-generated implementation and AI-generated tests. The development transcript contains several real examples where AI output was useful but incomplete. These were not invented after completion; they appeared while the test suite was being expanded.

5.1 Defect D1 - Greedy First-Fit Could Reject a Valid Timetable

What went wrong: the first working scheduler took the first legal room/time pair and never reconsidered it. All 100 tests then available passed, but a valid timetable could still be missed.

How it was found: after the suite was green, the algorithm was challenged with a case containing one large room, one small room, one time slot, a small class and a large class. Greedy first-fit placed the small class in the large room, leaving nowhere for the large class.

Fix: replace greedy first-fit with recursive depth-first backtracking. The algorithm now makes a choice, attempts to place the remaining sessions, and removes the choice if it leads to a dead end.

```
placed.append(Assignment(session, slot, room))
if self._place(order, index + 1, placed):
    return True
placed.pop()  # undo and try another option
```

Regression evidence: a new test produced 1 failed and 102 passed against the greedy version, then 103 passed after the backtracking fix.

5.2 Defect D2 - Prerequisite Rule Worked in Isolation but Failed in Search

What went wrong: the first prerequisite function only looked backwards at sessions already placed. This assumption was unsafe because the scheduler orders sessions by difficulty, not by timetable chronology.

How it was found: all seven direct prerequisite constraint tests passed, but two end-to-end scheduler tests still failed (114 passed, 2 failed). This showed that the rule itself appeared correct in isolation but was incomplete in the full search.

Fix: check prerequisite relationships in both directions on every placement. If an already-placed unit is my prerequisite, it must be earlier. If I am a prerequisite of an already-placed unit, I must also be earlier than that unit.

```
dependants = {unit.upper() for unit in prerequisites.get(earlier.session.unit_code, ())}
if session.unit_code in dependants and not slot.starts_before(earlier.slot):
    return False
```

Regression evidence: after the two-direction fix, all 116 tests passed.

5.3 Before-and-After Improvement Summary

The following comparison shows how AI-generated or AI-assisted implementations were changed after testing exposed a weakness. These are not hypothetical examples; they correspond to the failures documented in the development evidence.

Improvement	Before	After	Why the change was necessary
Greedy scheduling	Take the first legal room/time pair and never reconsider it.	Depth-first backtracking appends a choice, recursively tries the remaining sessions, and pops the choice when it leads to failure.	Prevents a locally legal early choice from falsely making a feasible timetable appear impossible.
Prerequisite checking	Only ask whether an already-placed unit is a prerequisite of the session currently being placed.	Check both directions for every already-placed session because search order is based on difficulty, not chronological order.	Prevents dependants from being placed before/simultaneously with prerequisites when the dependant happens to be searched first.
Coverage gap	Keep an unused Assignment.__str__ method and add a test only to execute it.	Remove the unused method as dead code.	Improves maintainability and makes 100% coverage reflect meaningful executable behaviour rather than artificial testing.

5.4 Other AI Output Improvements

- Numeric validation explicitly rejects bool values because isinstance(True, int) is True in Python.
- Overlap uses strict inequalities so back-to-back classes are allowed rather than treated as clashes.
- Soft preferences are implemented by sorting legal candidates, not filtering them out.
- The search step counter is reset between solve() calls so state does not leak from one run to the next.
- Deterministic tie-break keys were kept so the same input produces stable output.
- Assignment.__str__ was removed as dead code after coverage identified it as the only unexecuted statement; an artificial test was not added just to increase coverage.

5.5 Evaluation of the AI-Generated Tests

The test generation was generally strong because it included boundaries, invalid input and both positive and negative cases. However, the transcript also showed an important weakness: some tests passed against intentionally empty stub implementations. For example, a stub returning False naturally satisfies tests that expect a negative result. This means “test passed” is not enough by itself; paired tests and mutation of assumptions are needed to show that the test can actually detect an incorrect implementation.

The strongest tests were the regression and integration tests added after initially green states. The greedy regression test and the prerequisite scheduler tests both detected problems that ordinary unit tests had missed. These cases improved my testing strategy from simply checking functions to actively trying to falsify the algorithm.

5.6 Software Quality and Maintainability

Software quality was supported by design choices as well as by test quantity. The final code separates domain models, independent constraint functions and the search algorithm, which keeps individual behaviours testable and reduces coupling. Validation rejects malformed input early, while dedicated exceptions distinguish invalid specifications from a genuinely infeasible timetable. The backtracking search uses deterministic candidate ordering and a configurable step budget, improving reproducibility and protecting

against uncontrolled search. Soft constraints only rank already-legal candidates, so timetable quality can improve without weakening correctness. Regression tests remain in the suite for the greedy first-fit failure and prerequisite integration defect, preventing those defects from returning during later changes. The final coverage pass was also reviewed critically: the only uncovered statement was unused code, which was removed rather than tested artificially.

6. Testing Results

The supplied final project was executed again while preparing this report. The result was 134 passed tests. Running pytest-cov reported 262 executable statements in the timetable package with 0 missed statements (100% statement coverage). This result is supported by multiple test levels: low-level model validation, independent constraint-unit tests, end-to-end scheduler tests, boundary/invalid-input cases, regression tests for discovered defects, determinism checks and search-budget behaviour. Coverage is therefore presented as supporting evidence rather than as proof of correctness by itself.

```
python -m pytest -q
# 134 passed

python -m pytest --cov=timetable --cov-report=term-missing -q
# TOTAL 262 statements, 0 missed, 100%

python main.py
# Timetable found in 11 search steps
```

```
.....................................................................  [ 53%]
.....................................................                  [100%]

================================ tests coverage ================================
Name                         Stmts   Miss  Cover   Missing
----------------------------------------------------------
timetable/__init__.py            4      0   100%
timetable/constraints.py        23      0   100%
timetable/exceptions.py          4      0   100%
timetable/models.py            135      0   100%
timetable/scheduler.py          96      0   100%
----------------------------------------------------------
TOTAL                          262      0   100%

134 passed in 0.50s
```

Final testing/coverage evidence supplied with the project: 134 tests passing and 100% coverage.

The demo application also produced a valid timetable in 11 search steps. Its output assigned HIT137 and PRT582 sessions to legal rooms and times and reported a preferred-slot score of 1/1. The final timetable output was checked against room capacity, prerequisite ordering and lecturer availability rules.

6.1 Progress Across TDD Iterations

Iteration	RED result	GREEN/result	Main outcome
1	16 failed / 5 passed	21 passed	TimeSlot validation and overlaps
2	21 failed / 28 passed	49 passed	Room, Lecturer, Session
3	14 failed / 59 passed	73 passed	Independent hard constraints
4A	22 failed / 78 passed	100 passed	Initial greedy scheduler
4C	1 failed / 102 passed	103 passed	Greedy defect -> backtracking
5	7 failed / 109 passed; then 2 failed / 114 passed	116 passed	Prerequisite integration fix

6	4 failed / 121 passed	125 passed	Soft preferences and room efficiency
7	6 failed / 128 passed	134 passed	Budget, determinism, diagnostics
Final coverage	99% before cleanup	100%	Removed unused Assignment.__str__

7. Reflection

AI helped me most when the work was broken into small behaviours. Claude was good at proposing test cases, explaining why a boundary mattered, and generating small implementations after I had seen the tests fail. The constraint functions were a good example because each rule could be understood and tested separately before the scheduler tried to combine them.

The process also showed a problem with how I first used AI. My first request was too broad, and Claude responded by generating a complete project. That was convenient, but it did not match the assignment requirement to demonstrate iterative AI-TDD. I noticed this and asked to restart step by step. This changed the project from “AI gives me a solution” to “AI suggests the next test/implementation and I verify the result before continuing.”

The biggest lesson from TDD was that a green test suite does not mean the software is universally correct. The greedy scheduler reached 100 passing tests, but a carefully chosen new scenario still broke it. The same thing happened differently with prerequisites: every isolated prerequisite rule test passed while two end-to-end scheduler tests failed. These examples taught me that the quality and variety of tests matters more than only the number of tests.

I also learned that not every RED/GREEN cycle has to reveal a bug. Iteration 6 worked first time after the tests were added. Recording that honestly is better than inventing problems. Similarly, in Iteration 7 the coverage report found an unused __str__ method rather than a functional defect. I chose to delete the dead code instead of adding a meaningless test just to achieve 100% coverage.

The testing strategy improved the software by combining unit tests, boundary tests, invalid-input tests, regression tests and end-to-end scheduler tests. Unit tests made individual rules easy to diagnose, while the scheduler tests checked interactions between those rules and the search order. The regression tests now protect the project from returning to the greedy failure or one-direction prerequisite bug.

If I had more development time, I would test much larger randomly generated timetable problems, add performance benchmarks, explore constraint-propagation techniques before backtracking, support half-hour slots and room-equipment requirements, and add weighted lecturer preferences. I would also use continuous integration so every GitHub push automatically runs the full test and coverage suite.

Claude was the main AI tool used for the coding/TDD process. ChatGPT was later used to organise this final report from the assignment brief, final source code, Iterations.docx evidence and the supplied Claude conversation screenshots. I remain responsible for checking that the report matches the actual project and that I understand the submitted code and tests.

8. Repository and Submission Evidence

GitHub repository URL: [INSERT FINAL GITHUB REPOSITORY URL]

The final repository should contain the same version of the application and tests that produced the 134-pass, 100%-coverage result. The assignment submission ZIP should contain the final report, source code and automated unit tests.

The Claude transcript contains suggested git commit commands for each RED/GREEN stage, but the supplied ZIP does not contain verified commit evidence. Therefore the report does not claim that the complete commit

history already exists. Before submission, push the final project to GitHub and insert the repository URL above.

8.1 Evidence and Professional Presentation Checklist

Before final submission, the following items should be visible and consistent so that a marker can trace the report claims directly to the submitted artefacts:

- Insert the final Student ID on the cover page.
- Insert the final GitHub repository URL in Section 8 and confirm the repository contains the same code/tests used for the final evidence.
- Keep the RED/GREEN screenshots in chronological order with captions that identify the iteration and result.
- Include the final 134-passed/100%-coverage screenshot and the successful demo output.
- Ensure the ZIP contains this report, complete source code and automated unit tests as required by the brief.
- Do not claim historical Git commits unless those commits actually exist in the repository; use the genuine screenshots/transcript as process evidence where appropriate.

9. References

No external technical sources were supplied as evidence for this report. If Python, pytest, pytest-cov or other documentation was consulted during development, add those sources here in the referencing style required by the unit. The AI interactions are documented in the project development evidence rather than treated as authoritative technical references.

Appendix A - Evidence Map

The main report includes the complete sequence of 17 terminal screenshots from Iterations.docx. Their order corresponds to the RED/GREEN stages described in Section 4. The additional Claude conversation screenshots supplied during report preparation were used to reconstruct the actual interaction history, including the decision to restart the assignment iteratively, the greedy-scheduler defect discussion, the prerequisite integration defect, the soft-constraint design and the final coverage/dead-code decision.

Final verified commands executed while preparing this report:

```
python -m pytest -q                -> 134 passed
python -m pytest --cov=timetable --cov-report=term-missing -q -> 100%
python main.py                         -> timetable found in 11 search steps
```

BEFORE SUBMISSION: replace [INSERT STUDENT ID] and [INSERT FINAL GITHUB REPOSITORY URL]. Then re-run the final tests/coverage and ensure the submitted ZIP and GitHub repository match.